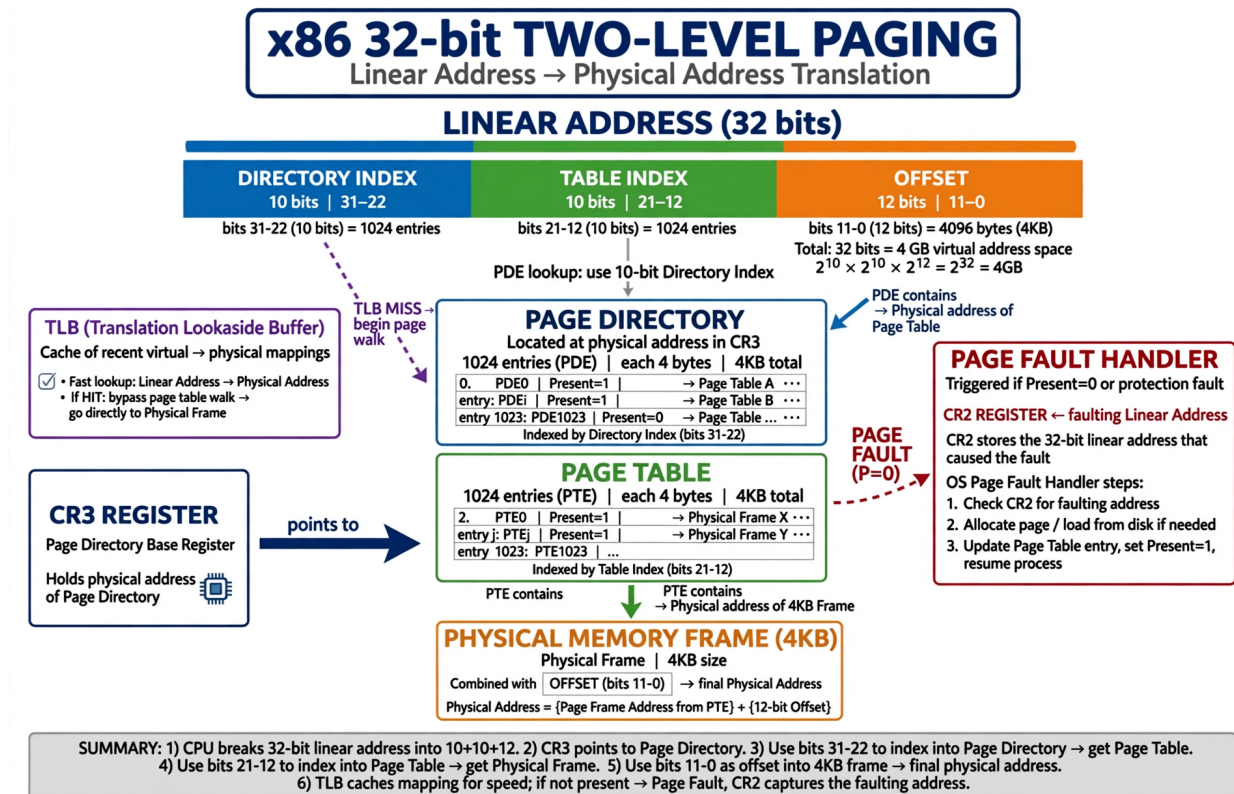


NanoOS/32 - Phase 2 - Memory Manager & Paging

Theory, Implementation and Bug-fix Commentary - Repo Documentation

Date: 2026-05-13 | Status: PHASE 2 PASSED - Paging Test Read 0x12345678 from 0xE0000000



1. Physical Memory Manager (PMM) - Theory

OS Theory: The BIOS/loader provides a memory map. PMM must track every 4KB physical frame as free/used. No malloc exists at this layer.

Bitmap method: 1 bit = 1 frame. 32MB = 8192 frames = 1024 bytes bitmap. Alloc = scan for 0-bit, set to 1, return frame*4096. Free = clear bit.

Implementation choice in NanoOS/32: Bitmap at 0x200000 (2MB). Why not 0x110000? Because kernel is loaded at 0x100000, .bss grows upward. Stack growing down from 0x110000 overwrote bitmap. This was the root cause of initial triple-fault: pmm_alloc_page() returned already-used frames, corrupting page tables.

Fix: kernel_entry.asm -> mov esp, 0x300FF0 (top of first 4MB identity region, safe). Bitmap at 0x200000 stays untouched.

2. Paging - x86 32-bit Two-Level

Linear address VA[31:0] split as DIR(10) | TABLE(10) | OFFSET(12).
DIR = VA >> 22 (0..1023). Example: 0xE0000000 >> 22 = 896.

TABLE = (VA >> 12) & 0x3FF.

OFFSET = VA & 0xFFF (byte inside 4KB frame).

CR3 holds physical address of Page Directory (1024 entries * 4 bytes = 4KB). PDE holds physical address of Page Table. PTE holds physical address of final 4KB frame + flags: P=Present (0x1), W=Writable (0x2), U=User (0x4).

Identity mapping theory: Before enabling PG bit in CR0, CPU runs with physical = linear. If you enable paging without identity mapping for kernel (0-4MB), the next instruction fetch would fault because EIP 0x100000 is no longer mapped. So Phase2 creates first_table mapping 0..0x3FFFFFF -> 0..0x3FFFFFF, and page_directory[0] = first_table|0x3.

3. TLB and invlpg - Why the single line matters

TLB (Translation Lookaside Buffer) caches VA->PA. When you modify page_directory[896] from 0 to present, TLB still remembers old "not present" entry for 0xE0000000. Without invalidation, next access still #PF even though tables are correct.

invlpg [m32] invalidates TLB entry for that linear address only. Alternative mov eax,cr3; mov cr3,eax flushes entire TLB (expensive). We use invlpg after every map_page/unmap_page.

GCC wrapper: .intel_syntax noprefix switches AT&T; to Intel syntax; .att_syntax prefix switches back; "memory" clobber tells compiler paging changed.

4. Page Fault (#PF, INT 14) - Theory and your log

CPU on #PF: 1) CR2 <- faulting VA, 2) push error_code, 3) jump via IDT[14]. Error code bits: bit0 P (0=not present,1=protection), bit1 W/R (0=read,1=write), bit2 U/S (0=supervisor,1=user).

Your log: PAGE FAULT at 0xE0000000 err=0x2 => 0b010 => P=0,W=1,S=0 => supervisor write to not-present page, which was exactly *(0xE0000000)=10.

IDT bug fix: IDT must be inside mapped memory. Initially IDT at 0x110000+something got corrupted by stack and after paging it was at unmapped VA. CPU on interrupt tried to read IDT -> #DF double-fault -> triple-fault reboot -> loader printed 'P'. Fix: IDT at 0x106060 (inside 0-4MB identity) and cli before mov cr0,0x80000000 to prevent timer IRQ during switch.

5. VMM - map_page() abstraction

VMM sits above PMM and Paging. PMM gives physical frames; VMM installs PTEs. Demand paging: allocate page table lazily.

Pseudocode:

dir = VA >> 22;

if PD[dir] not present: table = pmm_alloc_page(); zero 1024 entries; PD[dir]=table|0x3;

table = PD[dir] & 0xFFFFF000;

table[(VA >> 12) & 0x3FF] = (phys & 0xFFFFF000) | flags | 1;

invlpg VA;

Phase 2 test: frame = pmm_alloc_page(); map_page(0xE0000000, frame, 0x3); *(0xE0000000)=0x12345678; readback => 0x12345678 SUCCESS. Proves VMM works.

6. Bug Timeline (for future reference)

- Day1: Stack 0x110000 overlapped bitmap 0x200000? Actually 0x110000 overlap with kernel bss -> corrupted bitmap -> alloc returned kernel memory -> page tables overwritten -> triple fault on paging enable. - Day1 fix: esp=0x300FF0. - Day2: R flood -> kprintf loop in fault handler? Actually isr_common_stub pushed err_code as int_no. Fixed stub to handle #PF error code. - Day2: PAGE FAULT at 0xE0000000 err=0x2 but no halt -> missing cli/hlt loop. - Day2 fix: cli before paging, correct IDT base 0x106060, invlpg. - Day3: pmm_alloc_page() returns void* -> cannot | 0x3. Fix (uint32_t)pmm_alloc_page() | 0x3. - Day3 final: Mapping test PASSED.

7. Next Chapter - Task Switching (Theory Preview)

Memory gives isolation, scheduler gives concurrency. Each task needs: Task Control Block (TCB) with esp, ebp, eip, cr3 (its own page_directory!), kernel_stack, state.

TSS (Task State Segment): Intel hardware uses it for esp0 when switching from ring3->ring0. For software task switch we still need TSS with esp0 = current_task->kernel_stack_top.

Switch path: Timer IRQ 32 -> isr -> save current regs* onto its kernel stack -> current->esp = esp -> pick next task -> mov cr3, next->cr3 (switch address space) -> mov esp, next->esp -> popa -> iret -> next task resumes. Without PMM+VMM done correctly, per-task cr3 would map to garbage -> immediate fault.

Appendix A - Annotated Code

kernel_entry.asm - Stack fix

```
; Phase2 entry at 0x100000
[bits 32]
global _start
_start:
    ; THEORY: Stack grows DOWN. If esp=0x110000 and bitmap at 0x110000, first push overwrites bitmap.
    ; FIX: Put stack at top of identity-mapped low mem: 0x300FF0 = 3MB - 16, safe.
    mov esp, 0x300FF0

    cli                                ; THEORY: Disable IRQs before paging, timer could #PF during CR0 switch -> D
    call pmm_init                      ; Mark bitmap
    call paging_init                   ; Create PD[0]=identity 0-4MB
    call idt_init                      ; IDT at 0x106060 inside identity map!

    ; Enable paging
    mov eax, page_directory
    mov cr3, eax
    mov eax, cr0
    or eax, 0x80000000
    mov cr0, eax                      ; PG=1 here

    call kernel_main
    hlt
```

pmm.c - Annotated

```
#define BITMAP_BASE 0x200000
uint8_t *bitmap = (uint8_t*)BITMAP_BASE;
uint32_t total_frames;

void pmm_init() {
    // Parse BIOS memory map (from loader) - reserve 0-1MB, kernel 1-2MB, bitmap itself
    // Set all bits 1 = used, then clear usable RAM bits to 0 = free
}

void* pmm_alloc_page() { // returns physical address as void*
    // Linear scan for 0-bit -> mark 1 -> return i*4096
    // THEORY: First-fit, O(n). Later replace with stack/free-list.
    for(uint32_t i=0; i<total_frames; i++) {
        if(!(bitmap[i/8] & (1<<(i%8)))) {
            bitmap[i/8] |= (1<<(i%8));
            return (void*)(i*0x1000);
        }
    }
    return 0; // OOM -> should panic
}
```

paging.c - VMM map_page annotated

```
__attribute__((aligned(4096)))
uint32_t page_directory[1024];
uint32_t first_table[1024];

void map_page(uint32_t virt, uint32_t phys, uint32_t flags) {
    uint32_t dir_idx = virt >> 22; // 10 bits dir: 896 for 0xE0...
    uint32_t tbl_idx = (virt >> 12) & 0x3FF; // 10 bits table

    uint32_t pd_entry = page_directory[dir_idx];
    uint32_t *table;

    if(!(pd_entry & 0x1)) { // THEORY: demand allocate page table
        uint32_t new_tbl_phys = (uint32_t)pmm_alloc_page(); // PMM provides frame
        uint32_t *p = (uint32_t*)new_tbl_phys; // identity mapped, so virt==phys
        for(int i=0; i<1024; i++) p[i]=0x2; // mark all not present
        page_directory[dir_idx] = new_tbl_phys | 0x3; // PRESENT|WRITE
        table = p;
    }
```

```

    } else {
        table = (uint32_t*)(pd_entry & 0xFFFFF000);
    }
    table[tbl_idx] = (phys & 0xFFFFF000) | (flags & 0xFFF) | 0x1;

    // THEORY: TLB coherency
    __asm__ volatile(
        ".intel_syntax noprefix; invlpg dword ptr [%0]; .att_syntax prefix"
        :: "r"(virt) : "memory"
    );
}

```

idt + fault handler

```

struct regs { uint32_t edi,esi,ebp,esp,ebx,edx,ecx,eax; uint32_t int_no, err_code; ... };

void _fault_handler(struct regs *r) {
    uint32_t cr2;
    __asm__ volatile("mov %%cr2, %0" : "=r"(cr2));
    // r->int_no = 14, r->err_code = 2 for your test
    kprintf("PAGE FAULT at 0x%x err=0x%x\n", cr2, r->err_code);
    for(;;) __asm__ volatile("cli; hlt");
}

```

Document generated for NanoOS/32 repo /docs/Phase2_Memory_Manager.pdf - Keep with source.